Lab11 Tic-Tac-Toe

Source: MLL/Lab11 Tic-Tac-Toe.ipynb

CELL 1 — CODE | Execution: 12

```
import random
from collections import defaultdict

SEED = 42
random.seed(SEED)

WIN_LINES = [
    (0, 1, 2), (3, 4, 5), (6, 7, 8),  # Rows
    (0, 3, 6), (1, 4, 7), (2, 5, 8),  # Columns
    (0, 4, 8), (2, 4, 6)              # Diagonals
]

class TicTacToe:
    def __init__(self):
        self.board = [' '] * 9

    def reset(self):
        self.board = [' '] * 9
        return tuple(self.board)

    def available_moves(self):
        return [i for i, cell in enumerate(self.board) if cell == ' ']

    def make_move(self, position, player):
        if self.board[position] != ' ':
            raise ValueError("Invalid move: cell is occupied.")
        self.board[position] = player

    def check_winner(self, player):
        return any(all(self.board[i] == player for i in line) for line in WIN_LINES)

    def is_draw(self):
        return ' ' not in self.board and not self.check_winner('X') and not self.check_winner('O')

    def show(self):
        for i in range(0, 9, 3):
            print(' | '.join(self.board[i:i+3]))
            if i < 6:
                print('--+---+--')

class QLearningAgent:
    def __init__(self, alpha=0.1, gamma=0.9, epsilon=1.0):
        # Maps state tuple -> list of 9 Q-values
        self.Q = defaultdict(lambda: [0.0] * 9)
        self.alpha = alpha
        self.gamma = gamma
        self.epsilon = epsilon

    def get_q(self, state, action):
        return self.Q[state][action]

    def choose_action(self, state, available_actions, explore=True):
        if explore and random.random() < self.epsilon:
            return random.choice(available_actions)

        values = [self.get_q(state, a) for a in available_actions]
        max_value = max(values)
        best_actions = [a for a, v in zip(available_actions, values) if v == max_value]
        return random.choice(best_actions)

    def update(self, state, action, reward, next_state, next_actions, terminal=False):
        old_value = self.get_q(state, action)

        if terminal or not next_actions:
            future_value = 0.0
        else:
            future_value = max([self.get_q(next_state, a) for a in next_actions])

        target = reward + self.gamma * future_value
        self.Q[state][action] = old_value + self.alpha * (target - old_value)

def train_agent(episodes=20000, epsilon_start=1.0, epsilon_min=0.05, decay=0.9997):
    game = TicTacToe()
    agent = QLearningAgent(alpha=0.1, gamma=0.9, epsilon=epsilon_start)
    training_results = []

    for episode in range(episodes):
        state = game.reset()
        while True:
            avail_moves = game.available_moves()
```

```
            if not avail_moves:
                break

            # 1. Agent ('X') chooses and executes a move
            action = agent.choose_action(state, avail_moves)
            game.make_move(action, 'X')

            # Check if agent won or tied
            if game.check_winner('X'):
                agent.update(state, action, 1.0, tuple(game.board), [], terminal=True)
                training_results.append(1)
                break
            if game.is_draw():
                agent.update(state, action, 0.5, tuple(game.board), [], terminal=True)
                training_results.append(0)
                break

            # 2. Opponent ('O') makes a random move
            o_moves = game.available_moves()
            o_action = random.choice(o_moves)
            game.make_move(o_action, 'O')

            # Check if opponent won or tied
            if game.check_winner('O'):
                agent.update(state, action, -1.0, tuple(game.board), [], terminal=True)
                training_results.append(-1)
                break
            if game.is_draw():
                agent.update(state, action, 0.5, tuple(game.board), [], terminal=True)
                training_results.append(0)
                break

            # 3. Game continues: update Q-value for non-terminal state transition
            next_state = tuple(game.board)
            next_moves = game.available_moves()
            agent.update(state, action, 0.0, next_state, next_moves, terminal=False)
            state = next_state

        # Epsilon decay calculation
        agent.epsilon = max(epsilon_min, agent.epsilon * decay)

    return agent, training_results

# --- Execution ---
TOTAL_EPISODES = 20_000
agent, training_results = train_agent(episodes=TOTAL_EPISODES)

print("Training completed")
print(f"Training episodes: {TOTAL_EPISODES:,}")
print(f"Learned state-action values: {len(agent.Q):,}")
print(f"Final exploration rate (epsilon): {agent.epsilon:.4f}")
```

OUTPUT

```
Training completed
Training episodes: 20,000
Learned state-action values: 2,153
Final exploration rate (epsilon): 0.0500
```

CELL 2 — CODE

(Empty cell)